

Project Proposal: Smart Waste Management System

Liam Doherty - x25111175

This project supports UN Sustainable Development Goal 11: Sustainable Cities and Communities. The aim is to make cities more inclusive, safe, and sustainable. Efficient waste management systems are a critical component of urban sustainability, as it directly affects public health, environmental quality and optimisation of resources.

The proposed application is a smart waste management system, which simulates a distributed smart-city environment. The system consists of multiple connected services representing smart bins, a collection scheduling system and waste collection vehicles. This system is implemented using Node.js with gRPC while adhering to Protocol Buffers for service definitions.

These services communicate using gRPC to enable real-time coordination. The purpose of the application is to optimise waste collection by monitoring bin fill levels, prioritising pick ups, and dynamic coordination of collection routes. A graphical user interface (GUI) will be developed to allow users to discover services, monitor system activity and interact with the different components.

The system follows a distributed architecture where each service operates independently but collaborates through remote procedure calls. A naming service is also included to allow dynamic registration and discovery of services, fulfilling the requirement for service publication and discovery.

Bin monitoring service

This service represents smart waste bins across the city.

Functionalities:

- Retrieve the status of a bin (fill level, battery and overflow risk)
- Real-time overflow alerts
- Accept batch uploads of bin readings

Example RPCs:

- GetBinStatus(BinRequest) returns (BinStatus) (Unary RPC)
- **Request:** { binID: String }
- **Response:** { binId: string, fillLevel: int, batteryLevel: int, overflowRisk: bool }

- StreamOverflowAlerts(AreaRequest) returns (stream Alert) (Server Streaming RPC)

- **Request:** { area: string }
- **Response** (stream): {binId: string, message: string, timestamp: string}
- UploadBinReadings(stream BinReading) returns (UploadSummary) (Client Streaming RPC)
 - **Request** (stream): {binId: string, fillLevel: int, timestamp: string}
 - **Response:** { count: int, message: string }

Parameter Explanation:

- binId (string): Unique identifier for each bin
- fillLevel (int): Percentage fill level (0–100)
- batteryLevel (int): Remaining battery percentage
- overflowRisk (bool): Indicates if bin is near overflow
- timestamp (string): Time of reading in ISO format

Collection scheduling service

This service determines when bins should be collected based on incoming data.

Functionalities:

- Create and manage pickup requests
- Prioritise bins based on urgency
- Process batch reading from bins

Example RPC:

- CreatePickup(PickupRequest) returns (PickupResponse) (Unary RPC)
- **Request:** {binId: string, priority: int}
- **Response:** {pickupId: string, scheduledTime: string, status: string}

Parameter explanation:

Priority (int) Urgency level (from 1-5)

scheduleTime(string): planned collection time

Status (string): current state (eg scheduled, completed)

Fleet Control Service

This service manages communication with waste collection trucks.

Functionalities:

- Assign routes to trucks
- Track truck status & location
- Enable live communication with the control system

Example RPC:

- LiveTruckControl(stream ControlMessage) returns (stream TruckUpdate) (Bidirectional Streaming RPC)
- **Request** (stream): {truckId: string, command: string}
- **Response**(stream): {truckId: string, status: string, location: string}

Parameter Explanation:

- truckId (string): Unique identifier for each truck
- command (string): Instruction sent to truck (e.g., "start route")
- status (string): Current truck state
- location (string): Current GPS or area location

Naming Service

This service enables dynamic registration and discovery of services.

Functionalities:

- Register and unregister services
- Discover available services by type
- Provide a list of active services

Example RPC:

- RegisterService(ServiceInfo) returns (RegisterResponse)
- **Request:** { id: string, name: string, type: string, host: string, port: int }
- **Response:** {success: bool, message: string}

Parameter Explanation:

- id (string): Unique service identifier
- type (string): Service category (e.g., "bin", "fleet")
- host (string): Service IP or hostname
- port (int): Service port number

Use of gRPC communication

All communication between services is implemented using gRPC, enabling efficient and high performance remote procedure calls. The system demonstrated all four RPC types - unary, server streaming, client streaming and bidirectional streaming.

Together, these services form a cohesive distributed system that simulates a smart urban waste management environment. The **bin monitoring service** provides real time data, the **scheduling service** optimises operations and the fleet service executes collection tasks. The **naming service** ensures dynamic interaction between components. This integrated distributed system improves efficiency, reduces waste

overflow and directly contributes to achieving UN SDG 11 by promoting smarter and more sustainable urban waste management.